AI 辅助开发 · 方法论

用 AI 做一个真正能用的系统

从一句「我想做个工具」到稳定运行的产品，完整的 6 阶段流程，每阶段的重点事项与和 AI 协作的方式。

需求定义 | 架构设计 | 核心版本 | 迭代扩展 | 质量保障 | 交付维护

1 需求澄清 · 动手之前把问题想清楚

这一阶段最贵的不是时间，而是返工。花在这里的每一分钟，都能省去后面十倍的返工。

需求阶段

明确「为什么做」

用一句话说清楚：没有这个系统之前，是怎么做的？哪里最痛？有了之后什么会变？

- 场景：谁在用，什么时候用，在哪用
- 现在：现在是什么解决，哪里麻烦了
- 目标：用了系统之后，什么指标改变

让 AI 反过来问你

不是把需求告诉 AI，而是让 AI 追问，把你没想到的边界条件逼出来。

- 多人同时用吗？会不会冲突？
- 数据需要保存多久？谁能看？
- 有没有例外情况，特殊角色？
- 错误/报错又怎么处，还要做什么？

砍掉「想要但不必要」的功能

列出所有想要的功能，然后问自己：去掉哪些，这个系统还能用？留下的就是第一版。

- 必须有 / 应该有 / 可以有——三档分类，只做「必须有」

AI 协作的方式

你问：「我想做一个部门周报系统，有我想想我可能没考虑到的问题」

AI 问：「多少人同时填写？每没有权限区分，谁能改谁只能看？」

填完之后再要汇总给谁？数据需要保留多久？有没有考虑填错怎么办？」

你回答，这些回答是，**真正的需求就浮出来了**

⚠ 不要一上来就说「帮我写代码」——需求没想清楚就写的代码，一定要返工。

↓

2 架构设计 · 地基打好，上面才不歪

在写第一行代码之前，先把数据长什么样、用什么技术、系统有哪些模块定下来。

设计阶段

先定数据结构

把核心数据用最简单的 JSON 写出来，不需要完整，关键字段定下来就够。

```
{ "user": "张明",
  "role": "director",
  "week": "2026-W12",
  "tasks": [{"事项1"},{"事项2"}]
}
```

字段格式一旦定下来不要随便改，新旧数据格式混用是最常见的 bug 来源。

技术选型，只选一次

改技术选型的成本 = 重写所有代码。选之前先确认两件事：

- 国内用户能正常访问吗？（不能用 VPN）
- 依赖的服务三年后还在吗？
- 团队能维护吗？学习成本多高？

✅ 选稳定技术可行性，再写业务逻辑。

模块划分，做一职责

把系统拆成独立模块，每个模块只做一件事。

- 数据层：读写，格式化，云同步
- 权限层：谁能做什么操作
- 展示层：页面渲染，交互
- 通知层：提醒，导出，分享

⚠ 不要把业务逻辑写在 UI 代码里，后面一定后悔。

AI 协作的方式

你问：「我要做一个多人协作的周报系统，

数据存腾讯云 CloudBase，帮我设计数据结构和模块划分」

AI 给：数据字段定义 + 模块列表 + 潜在冲突点（如多端同时写入）

你确认：这步过一遍，有不合理的地方改一改**设计比迭代代码便宜** 10 倍

↓

3 最小可用版本 · 先跑起来，再说好不好

第一版的唯一目标是「能用」，不是「好看」「功能多」。跑起来才能发现真正的需求。

开发阶段

只做核心路径上的功能

用户完成一次完整操作需要哪几步？只做这几步，其他全砍掉。

- 登录 → 填写 → 保存 → 查看：这是核心路径
- 可视化看板、KPI评分、统计图表：第二版
- 导出 PDF、年度汇总、权限细分：第三版

✅ 每加一个功能先加一道复杂度，第一版复杂度要控量住。

建立改动规范，第一天就做

在项目里放一个 RULES.md，写三件事，从第一行代码开始执行：

- 命名规范：函数、变量、文件怎么命名
- 数据格式约定：日期用 ISO 8601，数值/字段单不混用
- 修改规则：改逻辑必须全局搜索同类逻辑一起改

⚠ 没有规范的代码每、三个月后连自己都看不懂。

每次让 AI 写代码之前，先说清楚影响范围

AI 只看到你给它的那段代码，它不知道「其他地方还有同样的逻辑」——除非你告诉它。

❌ 没有说明模式方式

⚠ 「把日期格式改成 MM/DD」

→ 只改了主函数，内部副本没改，两处格式不一致

✅ 正确的方式

✅ 「我要改日期格式，帮我先找所有处理日期的地方，然后一起改」

→ 一次改完，不漏隐患

AI 协作的方式

每次改动前问：「这个修改会影响哪些地方？改之前帮我列一遍」

遇到报错要求：「我说的 X，你理解错 → 还是 X？有没有其他可能？」

遇到反复要求：「背景是 #lazio，字号 13px，因为 App 一结真体版，不给版还说

↓

4 迭代扩展 · 一次只加一件事

第一版跑起来之后，用真实使用反馈驱动迭代。每次迭代只做一件明确的事。

迭代阶段

需求管理：先收集，再排序，再做

不要用产生一个新文件一个，每次只做第一条。排好优先级，一次性处理。

- 同一个功能的多个细节，放在一起说
- 区分「Bug 修复」和「新功能」
- 新功能上线前，先确认不破坏现有功能

加新功能前，审查数据兼容性

新功能可能需要给现有数据结构，用旧数据还在，会崩溃。

- 旧数据能被新逻辑正常读取吗？
- 需要写数据迁移脚本吗？
- 新旧字段格式是否一致？

⚠ 「新代码读不了老数据」是最难排查的 bug 类型。

每次迭代打 Git Tag

每次稳定版本打一个 tag，出问题随时能回到上一个能用的版本。

```
git tag v1.0-stable
git tag v2.0-bpi-feature
git push --tags
```

✅ 「上一个能用的版本」比「当前最新版本」更重要。

老代码有 AI 协作的方式

加新功能时：「我要加 X 功能，帮我先分析：

① 需要改哪些现有代码

② 会影响哪些已有功能

③ 数据格式需要改吗

分析完再动手。」

↓

5 质量保障 · 每次部署前的防线

不需要测试框架，只需要一张清单和一个习惯：部署之前过一遍，不过不发。

质检阶段

部署前 5 条必查清单

- 这次改动影响到的所有地方，都改了吗？
- 数据格式有新旧不兼容的地方吗？
- 有没有现代和旧格式同时产生 / 写入 / 读入？
- 旧数据能正常读取吗（用真实数据测）？
- 手机上能完整对吗？不只是桌面端？

✅ 跑 2 分钟过清单，省去 20 分钟线上返工。

发现 Bug 时的正确流程

- 先复现：能稳定复现的 bug 才能被修掉
- 定位范围：是数据问题，逻辑问题还是显示问题？
- 找根本原因：不是修症状，是修原因
- 确认影响面：修了这里，其他地方有没有同样的问题？
- 回归测试：修完后验证修之前能用的功能还能用

⚠ 修 bug 最容易「我觉得应该是这里，改了试试」——不定位就动手，会引入新 bug。

高频 BUG 时有 AI 协作的方式

不要说：「报错了，帮我修」（AI 只能猜）

要说：「出现了 X 问题，触发条件是 Y，

按钮信息是 Z，期望为可能是 A 或 B 造成的，帮我分析。」

AI 需要：完整报错信息 → 触发步骤 → 相关代码片段

信息越充整，定位越准，修改次数越少

↓

6 交付维护 · 上线不是终点

系统上线之后，真正的挑战才开始。维护成本往往比开发成本高，提前设计好。

维护阶段

保持文档和代码同步

每次改了重要逻辑，同步更新：

- 代码里的注释（解释「为什么」不是「是什么」）
- RULES.md 里的规格或代约定
- 关键决策的记录（为什么选这个方案）

✅ 三个月后的自己会感谢写注释的自己。

定期清理技术债务

每隔一段时间，专门花时间做一次：

- 删除没有用到的代码和变量
- 统一不一致的命名和格式
- 把重复出现的逻辑提取成函数
- 更新依赖版本，修复安全漏洞

⚠ 技术债务会复利增长，越晚处理越贵。

新功能上线前的标准流程

- 需求澄清（回到第 1 阶段）
- 评估对现有功能的影响
- 确认数据兼容性
- 先在本地测试，再部署
- 打 Git Tag 留档
- 告知使用者变化了什么

AI 协作的方式

现代开发中，AI 协作方式

现代开发中，「帮我审查这段代码，重点看：

① 有没有潜在的边界条件没处理

② 有没有重复逻辑可以抽取

③ 命名是否清晰」

很技术债清理：「这个文件上次改了很多，帮我找出

所有已经没用的代码和变量」

6 个阶段一览表

阶段	核心目标	重点事项	最容易犯的错
① 需求定义	把问题想清楚	明确场景和痛点，让 AI 追问边界条件，砍掉你必要功能	需求没想清楚就开始写代码
② 架构设计	打好地基	先定数据结构，技术选型只做一次，模块单一职责	边做边设计，改技术选型
③ 核心版本	先跑起来	只做核心路径，第一天建规范，改一处先搜全局	追求完美导致迟迟无法上线
④ 迭代扩展	一次加一件事	需求先收集再排序，审查数据兼容性，每次打 Tag	用产品什么就改什么，没有节奏
⑤ 质量保障	部署前过关	5 条清单必查，Bug 先定位再修，确认影响面	「应该没问题」直接上线
⑥ 交付维护	持续可用	文档和代码同步，定期清理债务，新功能走标准流程	只加功能不清理，系统越来越慢

「每一段都需要踩到才能知道，

这条路不是一遍是不知道坑在哪里的。」

走完一遍之后，这张图就是地图。

下次可以少踩八个。

华为集团研究院 · 系统开发经验沉淀 · 2025